

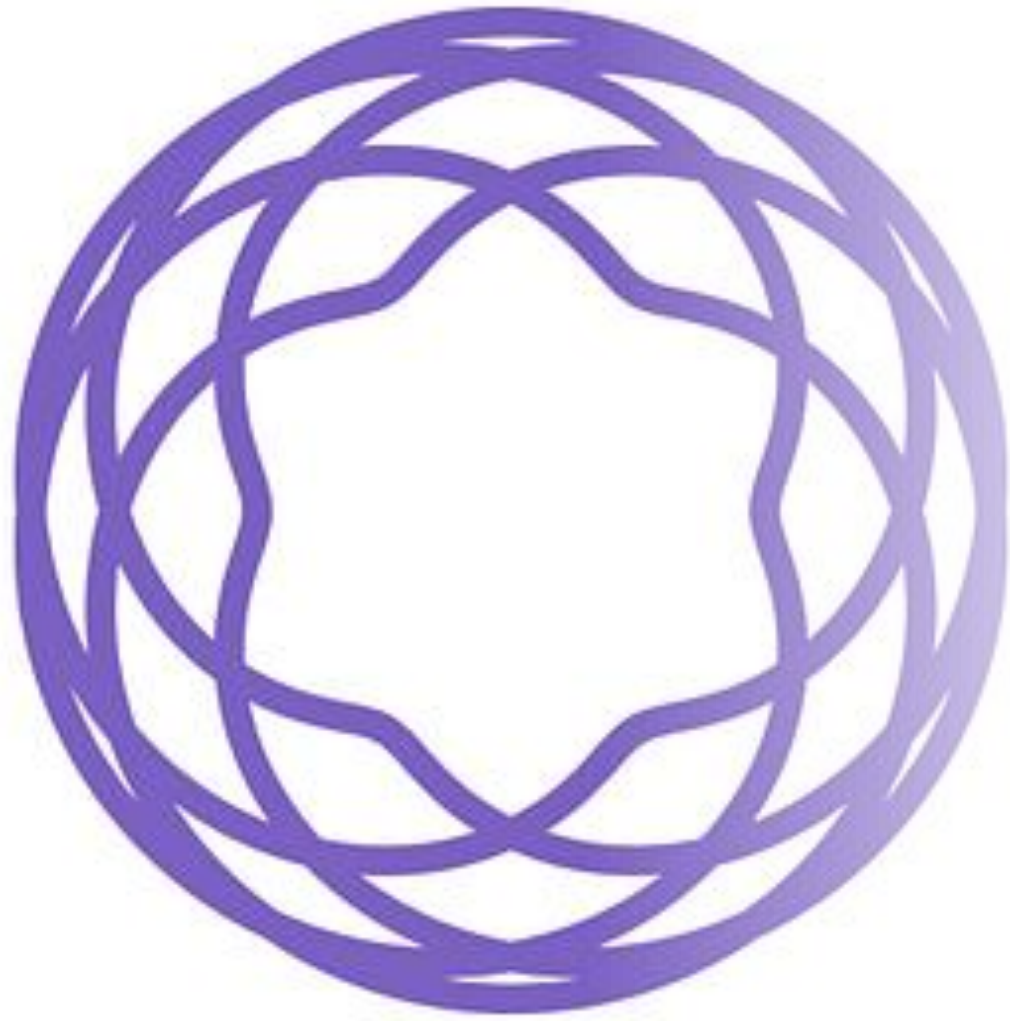


Asztali és mobil
alkalmazások fejlesztése
és tesztelése tantárgy –
Mobil alkalmazások rész

13. A

Tanárok: Dr. Koncz Adrienn
és Simon Martin

2026.03.19.



React Navigation

A bottom tab bar testreszabása

- Nézzük meg hogyan lehet testreszabni a tab bart a createBottomTabNavigator használatakor.
- Fontos, hogy mielőtt nekivágnánk a testreszabásnak, telepítsük a könyvtárat és konfiguráljuk megfelelően a BottomTab-ot a telepítési útmutató szerint.
- <https://reactnavigation.org/docs/customizing-tabbar>

Ikonok hozzáadása minden tabhoz

- Ez hasonló módon működik, mint amikor egy stack navigátort tesztelsz.
- Vannak olyan tulajdonságok, amelyeket a tab navigátor inicializálásakor állítasz be, és vannak olyanok, amelyeket képernyőnként külön az options beállításban adhatsz meg.
- Az ikonok listája itt található:
<https://ionic.io/ionicons/v4/cheatsheet.html>

```
// You can import Ionicons from @expo/vector-icons/Ionicons if you use Expo c
// react-native-vector-icons/Ionicons otherwise.
import Ionicons from 'react-native-vector-icons/Ionicons';

const RootTabs = createBottomTabNavigator({
  screenOptions: ({ route }) => ({
    tabBarIcon: ({ focused, color, size }) => {
      let iconName;

      if (route.name === 'Home') {
        iconName = focused
          ? 'ios-information-circle'
          : 'ios-information-circle-outline';
      } else if (route.name === 'Settings') {
        iconName = focused ? 'ios-list' : 'ios-list-outline';
      }

      // You can return any component that you like here!
      return <Ionicons name={iconName} size={size} color={color} />;
    },
    tabBarActiveTintColor: 'tomato',
    tabBarInactiveTintColor: 'gray',
  }),
  screens: {
    Home: HomeScreen,
    Settings: SettingsScreen,
  },
});
```

Az ikon nevébe
nem kell az ios

Mi történik a mintakódban?

- A `tabBarIcon` egy támogatott opció a bottom tab navigatorban. Ez azt jelenti, hogy használhatjuk a képernyő komponenseknél az `options` tulajdonságban. Ebben a példában azonban inkább a `screenOptions` tulajdonságban adtuk meg a `Tab.Navigator`-nél, hogy egy helyen, központilag kezeljük az ikonok konfigurációját.
- A `tabBarIcon` egy függvény, amely megkapja a `focused`, `color` és `size` paramétereket. Ha megnézed a konfiguráció későbbi részét, látni fogod a `tabBarActiveTintColor` és `tabBarInactiveTintColor` beállításokat. Ezek alapértelmezés szerint az iOS platform színeit használják, de itt szabadon módosíthatók.
- A `tabBarIcon` függvénynek átadott `color` érték attól függ, hogy a tab aktív vagy inaktív (a `focused` jelzi az aktív állapotot). A `size` pedig az ikon elvárt mérete, amelyet a tab bar használ.
- További részletekért és az összes konfigurációs lehetőségért nézd meg a `createBottomTabNavigator` teljes API dokumentációját.

Badge-ek hozzáadása az ikonokhoz

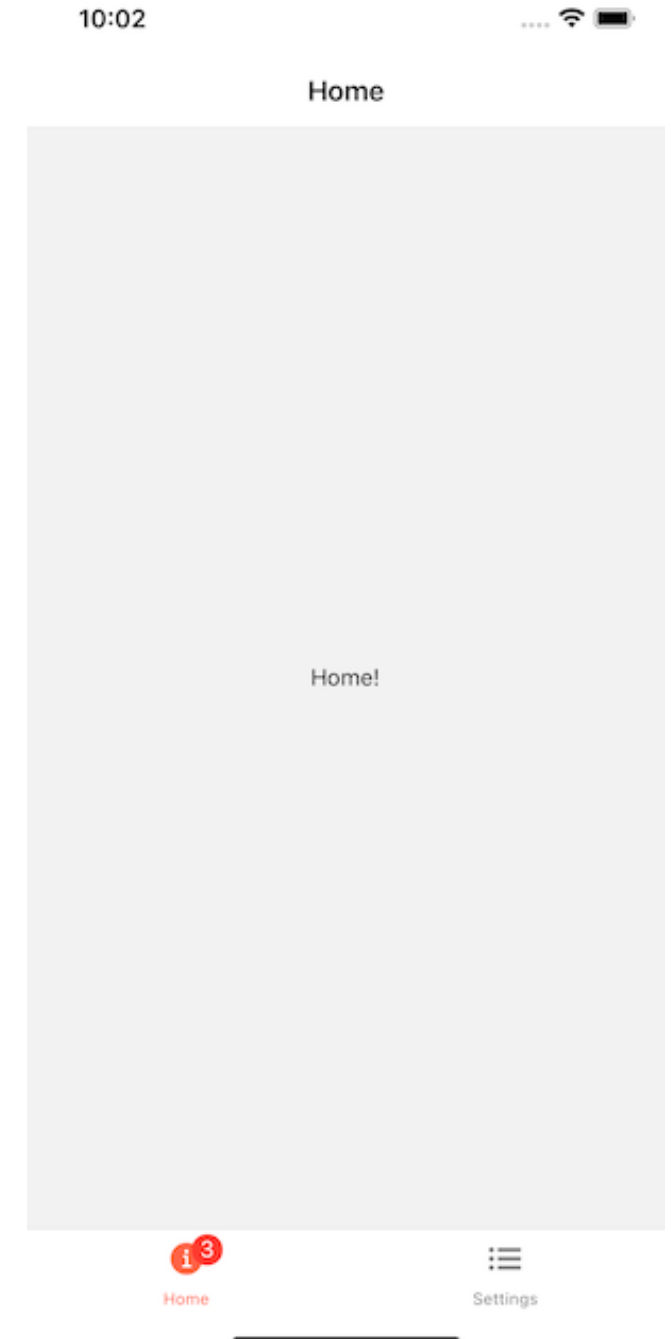
- Előfordulhat, hogy szeretnénk jelvényeket (badge-eket) megjeleníteni bizonyos ikonokon. Ezt a `tabBarBadge` opcióval tehetjük meg:

```
const RootTabs = createBottomTabNavigator({  
  screens: {  
    Home: {  
      screen: HomeScreen,  
      options: {  
        tabBarBadge: 3,  
      },  
    },  
    Settings: SettingsScreen,  
  },  
});
```

- Felhasználói felület szempontjából ez a komponens azonnal használható, azonban a badge számát valamilyen módon továbbra is át kell adnod az alkalmazás más részéből.
- Erre többféle megoldás létezik, például:
 - [React Context](#)
 - [Redux](#)
 - [MobX](#)
- Ezek segítségével lehet dinamikusan frissíteni a badge értékét az alkalmazás állapota alapján (például új üzenetek száma).
- A badge értékét közvetlenül a képernyő komponensből is frissítheted a [setOptions](#) metódus használatával:

```
const navigation = useNavigation();

React.useEffect(() => {
  navigation.setOptions({
    tabBarBadge: unreadMessagesCount,
  });
}, [navigation, unreadMessagesCount]);
```



A tab bar elrejtése bizonyos képernyőkön

- Előfordulhat, hogy egy tab navigátorba ágyazott stack navigátor bizonyos képernyőin el szeretnénk rejtetni a tab bart.
- Tegyük fel, hogy 5 képernyőnk van: Home, Feed, Notifications, Profile és Settings, és a navigációs struktúra így néz ki:

```
const HomeStack = createNativeStackNavigator({
  screens: {
    Home: Home,
    Profile: Profile,
    Settings: Settings,
  },
});

const MyTabs = createBottomTabNavigator({
  screens: {
    Home: HomeStack,
    Feed: Feed,
    Notifications: Notifications,
  },
});

const Navigation = createStaticNavigation(MyTabs);

export default function App() {
  return <Navigation />;
}
```

- Ezzel a struktúrával, amikor a Profile vagy a Settings képernyőre navigálunk, a tab bar továbbra is látható marad ezeken a képernyőkön.
- Ha viszont azt szeretnénk, hogy a tab bar csak a Home, Feed és Notifications képernyőkön jelenjen meg, de a Profile és Settings képernyőkön már ne, akkor át kell alakítanunk a navigációs struktúrát.
- Ennek legegyszerűbb módja, hogy nem a stack navigátort ágyazzuk a tab navigátorba, hanem a tab navigátort tesszük a stack első képernyőjébe.
- Miután átszerveztük a navigációs struktúrát, ha ezután a Profile vagy a Settings képernyőre navigálunk, a tab bar már nem fog látszani az adott képernyőkön.

```
const HomeTabs = createBottomTabNavigator({  
  screens: {  
    Home: Home,  
    Feed: EmptyScreen,  
    Notifications: EmptyScreen,  
  },  
});  
  
const RootStack = createNativeStackNavigator({  
  screens: {  
    Home: HomeTabs,  
    Profile: EmptyScreen,  
    Settings: EmptyScreen,  
  },  
});
```



Eltérő status bar beállítás útvonalanként

Ha nincs navigációs fejléced, vagy a navigációs fejléc színe az aktuális útvonaltól függően változik, akkor érdemes gondoskodni arról, hogy a tartalomhoz a megfelelő status bar szín legyen használva.

<https://reactnavigation.org/docs/status-bar>



Eltérő status bar beállítás útvonalanként – Stack

- Stack használata esetén ez egyszerű feladat. Egyszerűen kirenderelheted a React Native által biztosított StatusBar komponenst, és beállíthatod rajta a kívánt konfigurációt.
- Igazából ez kb. az a megoldás, mint amit külön tanultunk a StatusBar komponensnél.

```

import * as React from 'react';
import { View, Text, StatusBar, StyleSheet } from 'react-native';
import {
  createStaticNavigation,
  useNavigation,
} from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { Button } from '@react-navigation/elements';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

function Screen1() {
  const navigation = useNavigation();
  const insets = useSafeAreaInsets();

  return (
    <View
      style={[
        styles.container,
        {
          backgroundColor: '#6a51ae',
          paddingTop: insets.top,
          paddingBottom: insets.bottom,
          paddingLeft: insets.left,
          paddingRight: insets.right,
        },
      ]
    >
      <StatusBar barStyle="light-content" backgroundColor="#6a51ae" />
      <Text style={{ color: 'fff' }}>Light Screen</Text>
      <Button onPress={() => navigation.navigate('Screen2')}>
        Next screen
      </Button>
    </View>
  );
}

```

```

function Screen2() {
  const navigation = useNavigation();
  const insets = useSafeAreaInsets();

  return (
    <View
      style={[
        styles.container,
        {
          backgroundColor: '#ecf0f1',
          paddingTop: insets.top,
          paddingBottom: insets.bottom,
          paddingLeft: insets.left,
          paddingRight: insets.right,
        },
      ]
    >
      <StatusBar barStyle="dark-content" backgroundColor="#ecf0f1" />
      <Text>Dark Screen</Text>
      <Button onPress={() => navigation.navigate('Screen1')}>
        Next screen
      </Button>
    </View>
  );
}

```

Eltérő status bar beállítás útvonalként – Tab és Drawer

- Ha tab vagy drawer navigátort használasz, a helyzet egy kicsit bonyolultabb, mert a navigátorban lévő képernyők egyszerre is renderelve lehetnek, és renderelve is maradhatnak.
- Ez azt jelenti, hogy az utoljára beállított **StatusBar** konfiguráció lesz érvényben - valószínűleg nem azon a képernyőn, amit a felhasználó éppen lát, hanem például a tab navigátor utolsó tabján.
- Ennek javításához a status bar komponenst tudatosá kell tenni a képernyő fókuszára, és csak akkor szabad renderelni, amikor az adott képernyő fókuszban van.
- Ezt a **useIsFocused** hook segítségével érhetjük el, úgy, hogy létrehozunk egy burkoló komponenst:

```
import * as React from 'react';
import { StatusBar } from 'react-native';
import { useIsFocused } from '@react-navigation/native';

function FocusAwareStatusBar(props) {
  const isFocused = useIsFocused();

  return isFocused ? <StatusBar {...props} /> : null;
}
```

- Ezután a képernyőink (Screen1.js és Screen2.js) már nem közvetlenül a React Native StatusBar komponensét fogják használni, hanem a **FocusAwareStatusBar** komponenst.
- Bár nem kötelező, a **FocusAwareStatusBar** komponenst a native stack navigator képernyőin is használhatod.

```
function Screen1() {
  const navigation = useNavigation();
  const insets = useSafeAreaInsets();

  return (
    <View
      style={[
        styles.container,
        {
          backgroundColor: '#6a51ae',
          paddingTop: insets.top,
          paddingBottom: insets.bottom,
          paddingLeft: insets.left,
          paddingRight: insets.right,
        }
      ]}
    >
      <FocusAwareStatusBar barStyle="light-content" backgroundColor="#6a51ae" />
      <Text style={{ color: 'white' }}>Light Screen</Text>
      <Button onPress={() => navigation.navigate('Screen2')}>
        Next screen
      </Button>
    </View>
  );
}
```

```
function Screen2() {
  const navigation = useNavigation();
  const insets = useSafeAreaInsets();

  return (
    <View
      style={[
        styles.container,
        {
          backgroundColor: '#ecf0f1',
          paddingTop: insets.top,
          paddingBottom: insets.bottom,
          paddingLeft: insets.left,
          paddingRight: insets.right,
        }
      ]}
    >
      <FocusAwareStatusBar barStyle="dark-content" backgroundColor="#ecf0f1" />
      <Text>Dark Screen</Text>
      <Button onPress={() => navigation.navigate('Screen1')}>
        Next screen
      </Button>
    </View>
  );
}
```



Drawer Navigator

- A Drawer Navigator egy oldalsó navigációs menüt jelenít meg, amely a képernyő széléről húzható be, és érintéssel nyitható vagy zárható.
- A működését a [react-native-drawer-layout](#) könyvtár biztosítja.
- Ha a drawert React Navigation nélkül szeretnéd használni, akkor ezt a könyvtárat közvetlenül is használhatod.
- <https://reactnavigation.org/docs/drawer-navigator>

Import

- A drawer használatához ezt a könyvtárat kell importálni: @react-navigation/drawer

```
import { createDrawerNavigator } from '@react-navigation/drawer';

const MyDrawer = createDrawerNavigator({
  screens: {
    Home: HomeScreen,
    Profile: ProfileScreen,
  },
});
```

API definálás

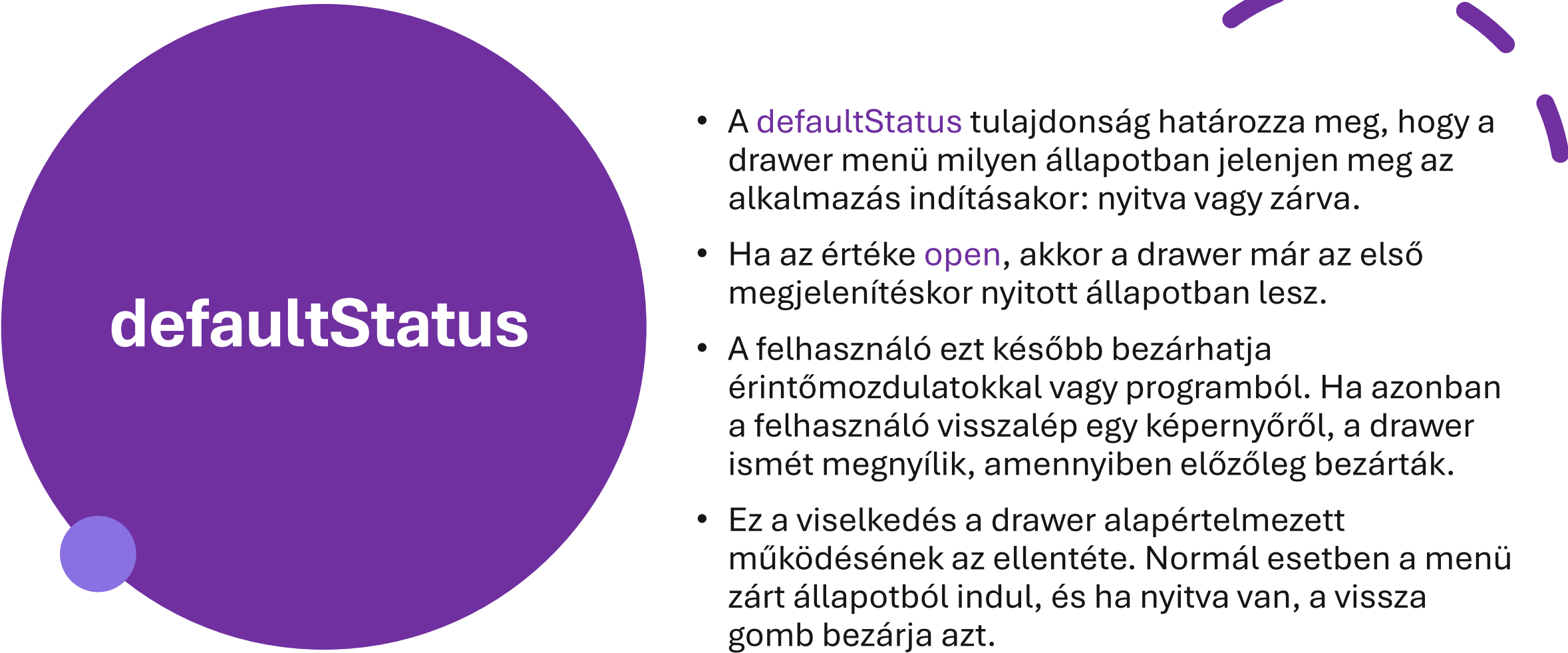
Props

- A minden navigátor által támogatott közös propokon felül a drawer navigator komponenssel a következő propok használhatók:
 - **backBehavior**



backBehavior

- A **backBehavior** tulajdonság azt határozza meg, hogy mi történjen akkor, amikor a felhasználó visszalép egy képernyőről. Ez bekövetkezhet például akkor, amikor a felhasználó megnyomja az Android eszköz vissza gombját, vagy egy vissza irányú érintőmozdulatot hajt végre.
- A backBehavior többféle működési módot támogat:
 - **firstRoute:** A navigátor visszatér az elsőként definiált képernyőre. Ez az alapértelmezett beállítás.
 - **initialRoute:** A navigátor visszatér az initialRouteName tulajdonságban megadott kezdőképernyőre. Ha ilyen nincs megadva, akkor az első képernyőre lép vissza.
 - **order:** A navigátor a jelenleg aktív képernyő előtti képernyőre lép vissza a navigációs sorrend alapján.
 - **history:** A navigátor arra a képernyőre tér vissza, amelyet a felhasználó legutóbb meglátogatott. Ha ugyanazt a képernyőt többször is megnyitják, akkor a régebbi bejegyzések törlődnek az előzményekből.
 - **fullHistory:** Szintén a legutóbb meglátogatott képernyőre lép vissza, azonban a history beállítással ellentétben nem törli a duplikált előzményeket. Ez a működés jobban hasonlít a weboldalak navigációs viselkedésére.
 - **none:** A navigátor nem kezeli a vissza gomb működését.

A large purple circle on the left side of the slide, with a smaller light purple circle at its bottom-left edge. In the top right corner, there are several purple curved lines and arcs of varying lengths.

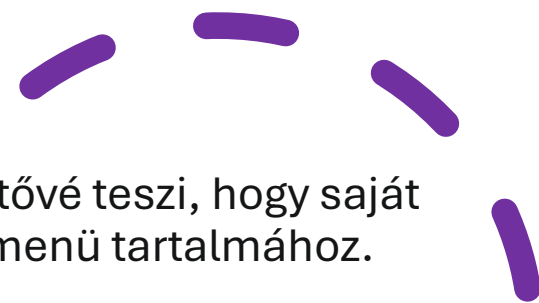
defaultStatus

- A `defaultStatus` tulajdonság határozza meg, hogy a drawer menü milyen állapotban jelenjen meg az alkalmazás indításakor: nyitva vagy zárva.
- Ha az értéke `open`, akkor a drawer már az első megjelenítéskor nyitott állapotban lesz.
- A felhasználó ezt később bezárhatja érintőmozdulatokkal vagy programból. Ha azonban a felhasználó visszalép egy képernyőről, a drawer ismét megnyílik, amennyiben előzőleg bezárták.
- Ez a viselkedés a drawer alapértelmezett működésének az ellentéte. Normál esetben a menü zárt állapotból indul, és ha nyitva van, a vissza gomb bezárja azt.



detachInactiveScreens

- A detachInactiveScreens egy logikai érték (*boolean*), amely azt határozza meg, hogy a nem aktív képernyők eltávolításra kerüljenek-e a nézethierarchiából.
- Ennek célja a memóriahasználat csökkentése, mivel az inaktív képernyők ilyenkor nem maradnak a felületi struktúrában.
- Ez a beállítás a navigátort a react-native-screens csomaggal integrálja. Alapértelmezett értéke true.



drawerContent

- A **drawerContent** tulajdonság lehetővé teszi, hogy saját komponenst adjunk meg a drawer menü tartalmához.
- Ez egy olyan függvény, amely egy React elemet ad vissza, és ez jelenik meg a drawer menüben. Itt például egyéni navigációs elemeket, fejléceket vagy más tartalmakat lehet megjeleníteni.
- A drawer tartalmát megjelenítő komponens alapértelmezés szerint a következő adatokat kapja:
 - **state**: A navigátor aktuális navigációs állapota.
 - **navigation**: A navigátorhoz tartozó navigációs objektum, amely lehetővé teszi a képernyők közötti navigációt.
 - **descriptors**: Egy objektum, amely a drawer képernyőkhöz tartozó konfigurációs beállításokat tartalmazza. Az egyes opciók így érhetők el: `descriptors[route.key].options`

Egyéni drawerContent használata

- A drawer alapértelmezett tartalma egy görgethető lista, amely a navigációs konfigurációban megadott képernyők linkjeit jeleníti meg.
- Ez az alapértelmezett komponens könnyen lecserélhető, ha például saját fejléct, lábléct vagy egyéb egyedi elemeket szeretnénk a menüben megjeleníteni.
- Az alapértelmezett drawer tartalom **DrawerContent** néven érhető el. Ez a komponens egy **DrawerItemList** elemet jelenít meg egy **ScrollView** belsejében.
- A drawer alapértelmezés szerint görgethető, és megfelelően kezeli a kijelző felső kivágásával (notch) rendelkező eszközöket is. Ha egyedi drawer tartalmat készítesz, használhatod a **DrawerContentScrollView** komponenst, amely ezt a működést automatikusan biztosítja.

```
import {
  DrawerContentScrollView,
  DrawerItemList,
} from '@react-navigation/drawer';

function CustomDrawerContent(props) {
  return (
    <DrawerContentScrollView {...props}>
      <DrawerItemList {...props} />
    </DrawerContentScrollView>
  );
}
```

- Ha további elemeket szeretnénk megjeleníteni a drawer menüben, használhatod a **DrawerItem** komponenst:

```
function CustomDrawerContent(props) {  
  return (  
    <DrawerContentScrollView {...props}>  
      <DrawerItemList {...props} />  
      <DrawerItem  
        label="Help"  
        onPress={() => Linking.openURL('https://mywebsite.com/help')}  
      />  
    </DrawerContentScrollView>  
  );  
}
```

☰ Home

Home Screen

Home

Settings

Help



DrawerItem

- A **DrawerItem** komponens segítségével egyedi menüpontokat adhatsz hozzá a drawerhez, például külön műveleteket, linkeket vagy egyedi navigációs elemeket.
- A **DrawerItem** komponens a következő tulajdonságokat támogatja:
 - **label** (kötelező): A menüpont felirata. Ez lehet egy egyszerű szöveg (string), vagy egy függvény, amely egy React elemet ad vissza.
 - Például:

```
{ { focused, color } } => <Text style={{ color }}>{focused ? 'Focused text' : 'Unfocused text'}</Text>
```

További DrawerItem tulajdonságok

- **icon:** Az elemhez tartozó ikon. Egy olyan függvényt vár, amely egy React elemet ad vissza.
 - Példa: `{ focused, color, size } => <Icon color={color} size={size} name={focused ? 'heart' : 'heart-outline'} />`
- **focused:** Logikai érték (*boolean*), amely azt jelzi, hogy a drawer elem aktív állapotban van-e, vagyis ki van-e emelve.
- **onPress (kötelező):** Az a függvény, amely akkor fut le, amikor a felhasználó rákattint az adott elemre.
- **activeTintColor:** Az ikon és a felirat színe, amikor az elem aktív állapotban van.
- **inactiveTintColor:** Az ikon és a felirat színe, amikor az elem nem aktív.
- **activeBackgroundColor:** Az elem háttérszíne aktív állapotban.
- **inactiveBackgroundColor:** Az elem háttérszíne inaktív állapotban.
- **labelStyle:** A feliratot megjelenítő Text komponens stílusát meghatározó objektum.
- **style:** Az elem külső View komponensének stílusát meghatározó objektum.

- Fontos megjegyezni, hogy a **useNavigation** hook nem használható a **drawerContent** komponensben, mert ez a hook csak képernyő komponenseken belül érhető el.
- A **drawerContent** azonban kap egy **navigation** nevű propot, amelyet ugyanúgy használhatsz navigációra:

```
function CustomDrawerContent({ navigation }) {  
  return (  
    <Button  
      onPress={() => {  
        // Navigate using the `navigation` prop that you received  
        navigation.navigate('SomeScreen');  
      }}  
    >  
      Go somewhere  
    </Button>  
  );  
}
```

- Ahhoz, hogy egy egyedi komponenst használjunk a drawer tartalmához, azt a **drawerContent** tulajdonságon keresztül kell átadnunk.

```
<Drawer.Navigator drawerContent={({props}) => <CustomDrawerContent {...props} />}>  
  {/* screens */}  
</Drawer.Navigator>
```



Options

- Ha megnézzük a weboldalon az Options részt, a felsorolt beállításokkal konfigurálhatók a navigátorban található képernyők. → <https://reactnavigation.org/docs/drawer-navigator#options>
- Ezeket a beállításokat kétféleképpen lehet megadni:
- a **Drawer.Navigator** komponens **screenOptions** tulajdonságában, ha minden képernyőre ugyanazt a beállítást szeretnénk alkalmazni;
- vagy egy adott képernyőnél a **Drawer.Screen** komponens **options** tulajdonságában, ha csak arra az egy képernyőre szeretnénk egyedi konfigurációt megadni.

Gyakorló feladatok

